

НИУ ИТМО
Факультет ПИиКТ

Лабораторная работа №1

Решение систем линейных алгебраических уравнений

Вариант №5

Преподаватель **Наумова Надежда Александровна**
Подготовил **Лоскутов Прохор Александрович**

Санкт - Петербург 2026

Оглавление

Оглавление	2
1 Вычислительная реализация задачи	3
1.1 Постановка задачи и обозначения	3
1.2 Прямой ход с выбором ведущего по столбцу	3
1.3 Распознавание трёх случаев	4
1.4 Обратный ход	4
1.5 Невязка	4
2 Программная реализация задачи	5
2.1 Блок-схема алгоритма	5
2.2 Исходный код алгоритма	5
2.3 Интерфейс приложения	8
3 Выводы	9

1 Вычислительная реализация задачи

По варианту №5 необходимо решить систему линейных алгебраических уравнений (СЛАУ) методом Гаусса с выбором главного элемента по столбцу. Для произвольной квадратной матрицы A размерности $n \times n$ и вектора свободных членов b требуется:

- привести расширенную матрицу $[A \mid b]$ к верхнетреугольному виду прямым ходом;
- вычислить определитель $\det A$ как произведение диагональных элементов с учётом знака перестановок;
- распознать три случая: единственное решение, отсутствие решений, бесконечное множество решений;
- в случае единственного решения найти неизвестные x_i обратным ходом;
- вычислить вектор невязок $r = A_0 x - b_0$, где A_0, b_0 — исходные (не преобразованные) данные.

1.1 Постановка задачи и обозначения

Рассматривается СЛАУ

$$\sum_{j=1}^n a_{ij} x_j = b_i, \quad i = 1, \dots, n.$$

В матричной форме: $Ax = b$. Расширенной матрицей называется $\tilde{A} = [A \mid b]$ размера $n \times (n + 1)$.

Элементарные преобразования строк сохраняют множество решений. Целью прямого хода является приведение A к верхнетреугольному виду U — все элементы ниже главной диагонали зануляются.

1.2 Прямой ход с выбором ведущего по столбцу

На шаге $i = 0, \dots, n - 1$:

1. Выбирается ведущая строка k среди $i \leq k < n$ такая, что $|a_{ki}|$ максимально. Это нужно для численной устойчивости: малые ведущие элементы катастрофически усиливают погрешность округления.

2. Если $|a_{ki}| < \varepsilon$, столбец i ниже текущей строки полностью нулевой — определитель обнуляется, шаг i пропускается.
3. Если $k \neq i$, строки i и k меняются местами; одновременно меняются местами компоненты вектора b , а знак перестановки sgn инвертируется.
4. Для всех $j = i + 1, \dots, n - 1$ вычисляется коэффициент $c = -\frac{a_{ji}}{a_{ii}}$, и к строке j прибавляется строка i , умноженная на c .

После завершения прямого хода имеем

$$\det A = \text{sgn} \cdot \prod_{i=0}^{n-1} a_{ii}.$$

1.3 Распознавание трёх случаев

После прямого хода:

- если $|\det A| \geq \varepsilon$, система имеет единственное решение;
- если $|\det A| < \varepsilon$ и существует строка i , в которой все $a_{ij} = 0$, но $b_i \neq 0$ — решений нет (последняя строка содержит несовместное уравнение вида $0 = c \neq 0$);
- если $|\det A| < \varepsilon$ и для всех строк с нулевыми коэффициентами также $b_i = 0$, система имеет бесконечно много решений (часть переменных свободна).

1.4 Обратный ход

Для несингулярной системы решение находится снизу вверх:

$$x_i = \frac{b_i - \sum_{j=i+1}^{n-1} a_{ij}x_j}{a_{ii}}, \quad i = n - 1, \dots, 0.$$

1.5 Невязка

Невязка — вектор разности фактической и заданной правой части:

$$r = A_0x - b_0.$$

Близость $\|r\|$ к нулю показывает численную точность найденного решения. В отличие от величины $\det A$ или числа обусловленно-

сти, невязка измеряет именно погрешность подстановки, а не теоретическую погрешность алгоритма.

2 Программная реализация задачи

2.1 Блок-схема алгоритма

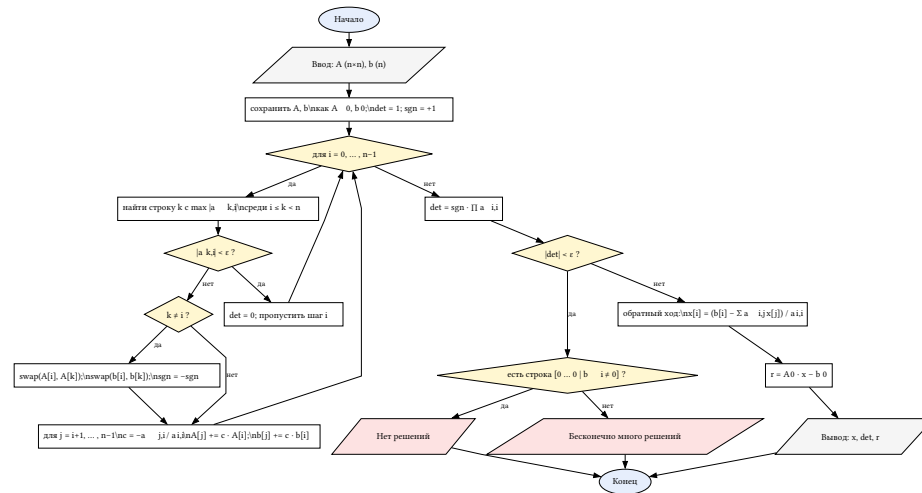


Рис. 1. Блок-схема метода Гаусса с выбором ведущего по столбцу

2.2 Исходный код алгоритма

Ядро метода Гаусса (файл `calc/gauss.cpp`):

```
#include "Gauss.hpp"
```

```
#include <algorithm>
```

```
#include <cmath>
```

```
#include <random>
```

```
#include <utility>
```

```
GaussResult
```

```
GaussSolver::solve(const std::vector<std::vector<double>> &inputMatrix,
                   const std::vector<double> &inputAugmentation) const {
```

```
    GaussResult result;
```

```
    const int n = static_cast<int>(inputMatrix.size());
```

```
    if (n ≤ 0 || n > kMaxSize ||
        static_cast<int>(inputAugmentation.size()) ≠ n) {
        result.status = GAUSS_INVALID_SIZE;
        return result;
    }
```

```
    for (const auto &row : inputMatrix) {
        if (static_cast<int>(row.size()) ≠ n) {
            result.status = GAUSS_INVALID_SIZE;
            return result;
        }
    }
```

```
    std::vector<std::vector<double>> matrix(inputMatrix);
    std::vector<double> augmentation(inputAugmentation);
```

```

std::vector<double> solution(n, 0.0);
std::vector<double> residuals(n, 0.0);

int swapSign = 1;
double det = 1.0;

for (int i = 0; i < n; ++i) {
    std::pair<double, int> maxVal = {matrix[i][i], i};
    for (int j = i + 1; j < n; ++j) {
        if (std::abs(maxVal.first) < std::abs(matrix[j][i])) {
            maxVal.first = matrix[j][i];
            maxVal.second = j;
        }
    }

    if (std::abs(maxVal.first) < kEps) {
        det = 0.0;
        continue;
    }

    if (maxVal.second != i) {
        std::swap(matrix[i], matrix[maxVal.second]);
        std::swap(augmentation[i], augmentation[maxVal.second]);
        swapSign = -swapSign;
    }

    for (int j = i + 1; j < n; ++j) {
        const double c = -matrix[j][i] / matrix[i][i];
        for (int k = i; k < n; ++k) {
            matrix[j][k] += c * matrix[i][k];
        }
        augmentation[j] += c * augmentation[i];
    }
}

for (int i = 0; i < n; ++i) {
    det *= matrix[i][i];
}
det *= swapSign;

result.triangular = matrix;
result.reducedAugmentation = augmentation;
result.determinant = det;

if (std::abs(det) < kEps) {
    for (int i = 0; i < n; ++i) {
        bool allZeros = true;
        for (int j = 0; j < n; ++j) {
            if (std::abs(matrix[i][j]) > kEps) {
                allZeros = false;
                break;
            }
        }
        if (allZeros && std::abs(augmentation[i]) > kEps) {
            result.status = GAUSS_NO_SOLUTIONS;
            return result;
        }
    }
    result.status = GAUSS_INFINITE_SOLUTIONS;
    return result;
}

```

```

}

for (int i = n - 1; i ≥ 0; --i) {
    double sum = 0.0;
    for (int j = i + 1; j < n; ++j) {
        sum += matrix[i][j] * solution[j];
    }
    solution[i] = (augmentation[i] - sum) / matrix[i][i];
}

for (int i = 0; i < n; ++i) {
    double sum = 0.0;
    for (int j = 0; j < n; ++j) {
        sum += solution[j] * inputMatrix[i][j];
    }
    residuals[i] = sum - inputAugmentation[i];
}

result.status = GAUSS_SINGLE_SOLUTION;
result.solution = solution;
result.residuals = residuals;
return result;
}

std::vector<std::vector<double>>
GaussSolver::generateMatrix(int n, unsigned long long seed) {
    std::mt19937_64 gen(seed);
    std::uniform_real_distribution<double> urd(0.0, 10.0);
    std::vector<std::vector<double>> res(n, std::vector<double>(n));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            res[i][j] = urd(gen);
        }
    }
    return res;
}

std::vector<double> GaussSolver::generateAugmentation(int n,
                                                       unsigned long long seed) {
    std::mt19937_64 gen(seed ^ 0x9E3779B97F4A7C15ULL);
    std::uniform_real_distribution<double> urd(0.0, 10.0);
    std::vector<double> res(n);
    for (int i = 0; i < n; ++i) {
        res[i] = urd(gen);
    }
    return res;
}

```

Заголовочный файл calc/Gauss.hpp:

```

#pragma once

#include <vector>

enum GaussStatus {
    GAUSS_SINGLE_SOLUTION = 0,
    GAUSS_NO_SOLUTIONS = 1,
    GAUSS_INFINITE_SOLUTIONS = 2,
    GAUSS_INVALID_SIZE = 3
};

```

```

struct GaussResult {
    GaussStatus status = GAUSS_SINGLE_SOLUTION;
    std::vector<std::vector<double>> triangular;
    std::vector<double> reducedAugmentation;
    std::vector<double> solution;
    std::vector<double> residuals;
    double determinant = 0.0;
};

class GaussSolver {
public:
    static constexpr double kEps = 1e-12;
    static constexpr int kMaxSize = 20;

    GaussResult solve(const std::vector<std::vector<double>> &matrix,
                     const std::vector<double> &augmentation) const;

    static std::vector<std::vector<double>>
    generateMatrix(int n, unsigned long long seed);

    static std::vector<double> generateAugmentation(int n,
                                                    unsigned long long seed);
};

```

2.3 Интерфейс приложения

UI-вкладка «Метод Гаусса» (первая в навигации) состоит из двух колонок: слева — ввод размерности n (от 1 до 20), редактируемая матрица $[A \mid b]$ и кнопки «Случайные коэффициенты» / «Вычислить»; справа — статус, определитель $\det A$, треугольная матрица после прямого хода, найденные неизвестные x_i и вектор невязок r_i .

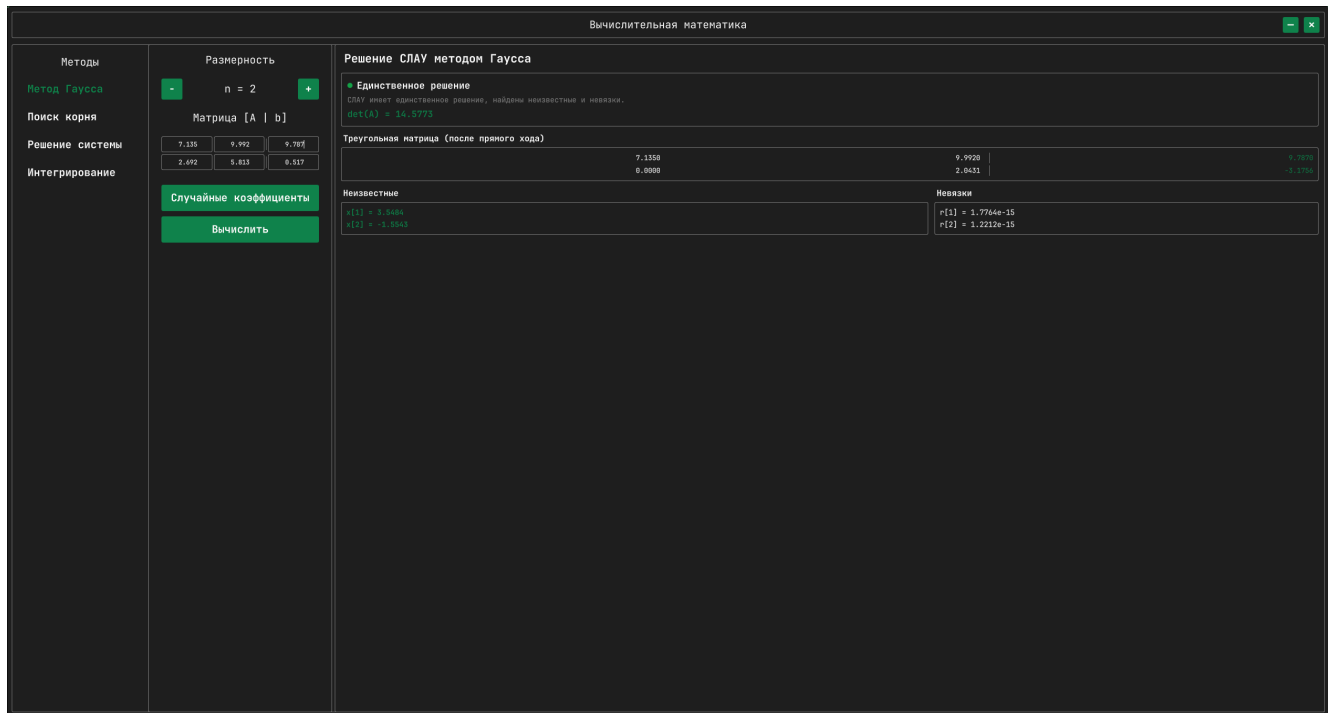


Рис. 2. Внешний вид вкладки «Метод Гаусса» в приложении

3 Выводы

В ходе лабораторной работы реализован численный метод решения систем линейных алгебраических уравнений — метод Гаусса с выбором ведущего элемента по столбцу. Алгоритм корректно различает три случая (единственное решение, отсутствие решений, бесконечное множество), параллельно вычисляет определитель матрицы коэффициентов и вектор невязок $r = A_0x - b_0$, который служит численной мерой точности найденного решения.

Программная часть оформлена как самостоятельная C++ библиотека `GaussSolver` без зависимостей от Qt, что позволяет переиспользовать её в других контекстах (например, как часть прямого решателя для метода Ньютона из ЛР №2). Qt-обёртка `GaussModule` отвечает за маршалинг данных между QML и C++, а вкладка `ModuleGauss.qml` — за пользовательский ввод произвольной матрицы размерности до 20×20 и наглядное представление трёх видов результата (треугольная матрица, решение, невязки). Выбор ведущего по столбцу обеспечивает численную устойчивость алгоритма даже при больших разбросах элементов исходной матрицы — это согласуется с поведением реализации на случайно сгенерированных тестах.